Université Paris Nanterre - L2 MIASHS

UE Programmation Web - 2026

CINÉCOLLECTION

Conception et Développement d'une Médiathèque Web Dynamique

Version 2

Anasse BOUYDARNE & Abdelkader CHERFAOUI

Version d'origine réalisée en binôme · Version 2 reprise et enrichie par Anasse BOUYDARNE

Version d'origine sous la direction de : M. NAFI & Mme. HEDJAJ

1. Introduction et Sujet du Projet

Le projet **CinéCollection** s'inscrit dans le cadre de l'UE de Programmation Web de notre deuxième année de licence MIASHS. L'objectif était de concevoir, développer et déployer une application web dynamique complète (architecture Client-Serveur) permettant à un utilisateur de gérer sa propre collection de films de manière interactive et sécurisée.

Le sujet est né d'un constat simple : il manquait un outil personnel, sans la lourdeur des réseaux sociaux spécialisés, permettant d'archiver ses visionnages, d'attribuer des notes et d'obtenir des statistiques sur ses propres goûts cinématographiques.

La première version, rendue en binôme, proposait une connexion simple et une gestion CRUD des films de chaque utilisateur. Cette **version 2** reprend le projet pour en faire une véritable plateforme : base de données relationnelle complète, comptes avec email et rôles, listes de films, avis, commentaires, collections publiques et espace d'administration.

Ce qui change entre les deux versions

Domaine	Version 1	Version 2
Base de données	2 tables (users, films)	8 tables reliées, catalogue de films partagé
Comptes	Pseudo et mot de passe	Email, mot de passe robuste, « Se souvenir de moi », profil, rôles membre / admin
Collection	Un film, une note	Statut vu / à voir, favoris, note, avis, date de visionnage, plusieurs genres
Partage	Aucun	Collections publiques, page Membres, commentaires
Administration	Aucune	Statistiques globales, gestion des membres, des genres et du catalogue
Sécurité	PDO, XSS, hachage	+ jetons CSRF, limite des essais de connexion, jetons de session hachés

2. Architecture et Environnement Technique

Le projet conserve une architecture Full-Stack classique, organisée selon une logique proche du modèle MVC (Modèle-Vue-Contrôleur). La version 2 sépare davantage les responsabilités : un

fichier d'amorçage commun, des fonctions d'authentification et des fonctions utilitaires réunies dans un dossier `inc/`, et une page PHP par écran.

> - **Front-End (Client) :** HTML5 sémantique, CSS3 (variables, Grid Layout, responsive mobile) et JavaScript.
> - **Back-End (Serveur) :** PHP 8, sessions, PDO.
> - **Base de Données :** MySQL / MariaDB, encodage utf8mb4, moteur InnoDB.
> - **Environnement de déploiement :** serveur local Apache via XAMPP.
> - **Gestion de version :** Git et GitHub. Le premier commit correspond à la version rendue, les suivants à la version 2.

Organisation des fichiers

Fichier	Rôle
`inc/bootstrap.php`	Configuration, connexion PDO, session sécurisée, connexion automatique
`inc/auth.php` · `inc/helpers.php`	Authentification, rôles, « Se souvenir de moi », CSRF, échappement, validations
`index.php` · `film.php`	Collection avec filtres et statistiques, ajout et modification d'un film
`login.php` · `register.php` · `profil.php`	Connexion, inscription, gestion du compte
`membres.php` · `collection.php`	Collections publiques, avis et commentaires
`admin.php` · `actions.php`	Administration, actions rapides (favori, commentaire, suppression)

3. Conception Front-End : Interfaces et UX

3.1. Design et Sémantique

L'interface s'inspire des plateformes de streaming : thème sombre, grille d'affiches, badges de statut et cœur pour les favoris. Les balises sémantiques (`<header>`, `<nav>`, `<main>`, `<article>`) structurent chaque page. La grille utilise `display: grid` avec `repeat(auto-fill, minmax(...))` pour s'adapter à toutes les tailles d'écran, et un menu repliable prend le relais sur mobile.

Lorsqu'un film n'a pas d'affiche, l'application génère une affiche de remplacement avec le titre et l'année, dont la couleur est calculée à partir du titre : la grille reste lisible et variée.

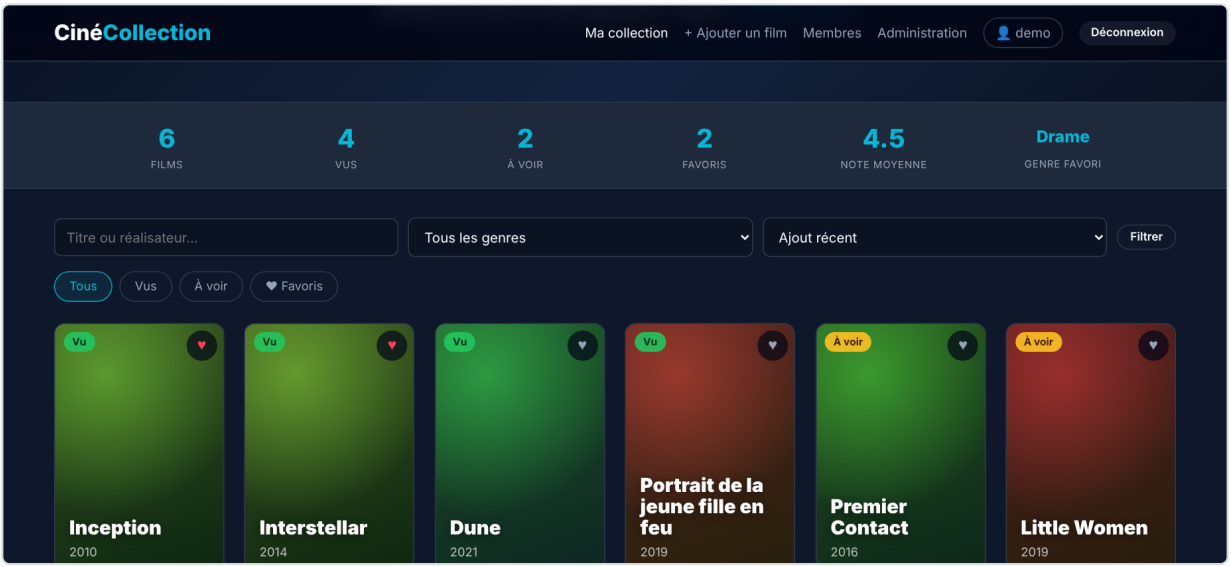


Figure 1 : la collection d'un membre, avec statistiques, filtres et onglets

3.2. Navigation et Interactions

- **Filtres et tri :** recherche par titre ou réalisateur, filtre par genre, onglets Tous / Vus / À voir / Favoris, tri par date d'ajout, titre, année ou note.
- **Formulaire de film :** genres en cases à cocher, notation par étoiles en CSS pur, champs note et date masqués automatiquement pour un film « À voir ».
- **Retours utilisateur :** messages de confirmation et d'erreur affichés en notifications, demande de confirmation avant toute suppression, indicateur de force du mot de passe.

4. Modélisation de la Base de Données

La version 1 reposait sur deux tables. La version 2 passe à un modèle relationnel de **8 tables**. Le choix principal est de séparer le **catalogue de films**, partagé entre tous les membres, de la **collection** de chaque membre : un même film n'est enregistré qu'une fois, et chaque membre y associe ses propres informations.

Table	Contenu
`users`	Comptes : pseudo et email uniques, mot de passe haché, rôle (user / admin), bio, collection publique ou non
`remember_tokens`	Jetons « Se souvenir de moi » : identifiant public et hash SHA-256 du secret, date d'expiration
`films`	Catalogue : titre et année (couple unique), affiche, synopsis, réalisateur, membre ayant ajouté le film

`realisateurs` · `genres`	Tables de référence, noms uniques
`film_genres`	Table d'association : un film a plusieurs genres, un genre concerne plusieurs films (relation N-N)
`collection`	Lien membre ↔ film : statut, favori, note (contrainte CHECK entre 1 et 5), avis, date de visionnage
`commentaires`	Réponses des membres sous un avis

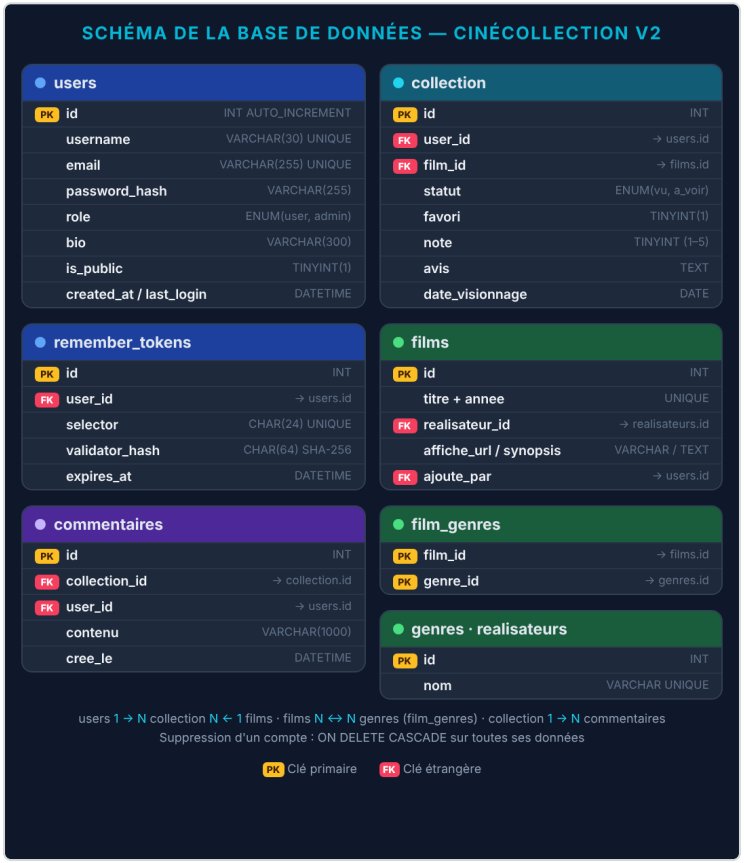


Figure 2 : schéma relationnel de la version 2

Intégrité référentielle : toutes les relations sont des clés étrangères. `ON DELETE CASCADE` supprime la collection, les jetons et les commentaires d'un membre avec son compte, tandis que `ON DELETE SET NULL` conserve un film du catalogue même si le membre qui l'a ajouté disparaît. Les contraintes `UNIQUE` empêchent les doublons (un film en double dans le catalogue, ou deux fois dans la même collection).

5. Logique Serveur et Fonctionnalités (Back-End)

5.1. Comptes et rôles

- **Inscription :** pseudo (3 à 30 caractères), email valide et unique, mot de passe d'au moins 8 caractères avec une lettre et un chiffre, confirmé deux fois. Le premier compte créé devient administrateur.
- **Connexion :** par pseudo ou par email. L'option « Se souvenir de moi » garde le membre connecté 30 jours.
- **Profil :** modification du pseudo, de l'email et de la bio, changement de mot de passe (qui déconnecte les autres appareils), déconnexion de tous les appareils, suppression du compte après confirmation par mot de passe.
- **Rôles :** un visiteur ne voit que les collections publiques, un membre gère sa collection et commente, un administrateur accède en plus au tableau de bord.

5.2. Collection, partage et administration

L'ajout d'un film vérifie d'abord s'il existe déjà dans le catalogue (même titre et même année) : si oui, il est simplement ajouté à la collection du membre. Seul le membre qui a ajouté un film au catalogue, ou un administrateur, peut modifier ses informations communes ; chacun reste libre de modifier ses informations personnelles. Ces opérations s'exécutent dans une **transaction** pour garder la base cohérente en cas d'erreur.

Chaque membre peut rendre sa collection publique. Les autres membres peuvent alors lire ses avis, y répondre par un commentaire et ajouter un film à leur propre liste « À voir » en un clic. L'administrateur dispose de statistiques globales et peut changer le rôle d'un membre, supprimer un compte, gérer les genres et retirer un film du catalogue.

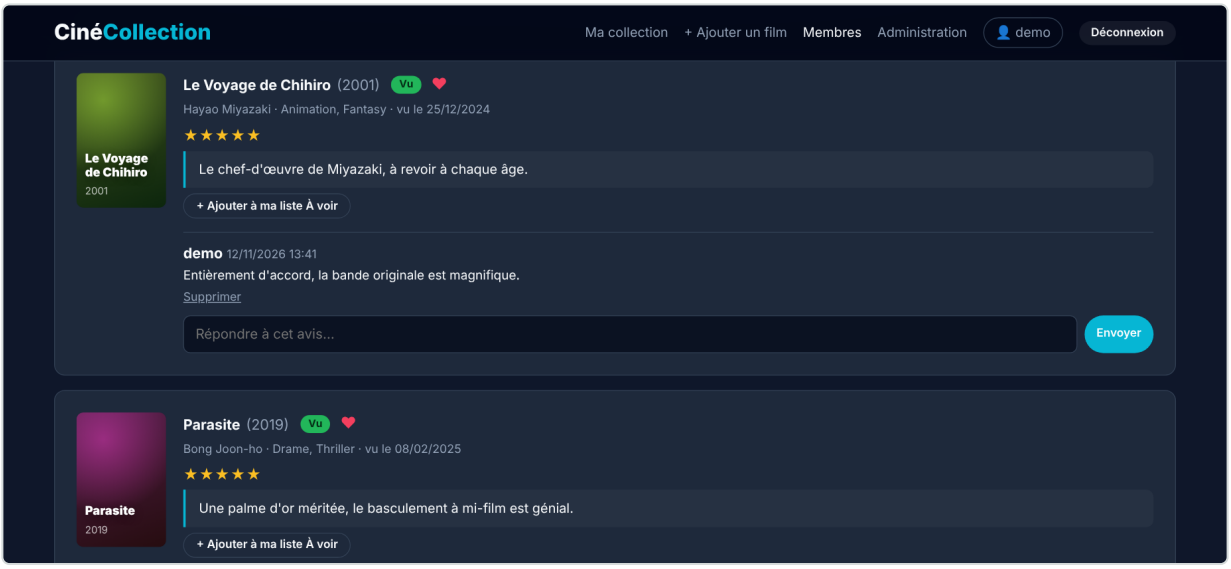


Figure 3 : page publique d'un membre, avec avis et commentaires

5.3. Statistiques

Les statistiques sont calculées directement en SQL avec des fonctions d'agrégation (`COUNT`, `SUM`, `AVG`) et des regroupements (`GROUP BY`, `GROUP_CONCAT` pour afficher les genres d'un film), plutôt qu'en parcourant les données en PHP.

6. Sécurisation de l'Application

La sécurité reste le fil rouge du projet. La version 2 conserve les protections de la version 1 et en ajoute de nouvelles :

1. **Requêtes préparées (PDO) :** aucune saisie n'est concaténée dans une requête SQL, ce qui bloque les injections SQL. L'émulation des requêtes préparées est désactivée.
2. **Protection XSS :** toutes les données affichées passent par `htmlspecialchars()`. Les URL d'affiche ne sont acceptées que si elles commencent par `http://` ou `https://`.
3. **Protection CSRF :** chaque formulaire envoie un jeton aléatoire stocké en session, vérifié côté serveur avec `hash_equals()`. Toutes les actions qui modifient des données, y compris la déconnexion, passent par POST.
4. **Mots de passe :** hachés avec `password_hash()`, vérifiés avec `password_verify()`, et re-hachés automatiquement si l'algorithme évolue.
5. **Sessions :** cookies `HttpOnly` et `SameSite=Lax`, identifiant de session régénéré à la connexion, limite de 5 essais de connexion avant un blocage de 5 minutes.
6. **« Se souvenir de moi » :** le cookie contient un identifiant et un secret. Seul le hash du secret est stocké en base, et le jeton est remplacé à chaque utilisation.
7. **Contrôle d'accès :** chaque requête qui touche une collection vérifie que l'élément appartient bien au membre connecté (`WHERE id = ? AND user_id = ?`). Les identifiants MySQL sont dans `config.php`, exclu du dépôt GitHub.

7. Conclusion et Bilan

Le projet CinéCollection a été une expérience extrêmement formatrice. La première version nous a permis de faire dialoguer les différents langages étudiés (HTML, CSS, JavaScript, PHP, SQL) au sein d'une même architecture Full-Stack, et de découvrir les réalités d'un travail en binôme.

La version 2 montre comment une application évolue : une modélisation plus riche de la base de données, une gestion des comptes plus complète, des fonctionnalités sociales et une sécurité renforcée. Elle souligne l'importance de bien concevoir la base en amont, puisque presque toutes les nouvelles fonctionnalités en découlent.

L'application est fonctionnelle, sécurisée et déployable sur n'importe quel serveur Apache / MySQL. Le code source, les instructions d'installation et un jeu de données de démonstration sont disponibles sur **github.com/Anasse95/cinecollection**.